

Cracking the OWASP 10 for Kubernetes



What is OWASP?

- The [Open Worldwide Application Security Project](#) (OWASP) is a non-profit foundation dedicated to improving software security through free, open-source resources, tools, and community-led projects
- The **"Top 10" Standard**: Maintains the industry-standard list of the most critical web application security risks.

K01: Insecure Workload Configurations

Description

- By default, Kubernetes workloads run with weak security controls, which can increase the risk of application compromise and lateral movement across the cluster.
- Production workloads should apply **securityContext** settings and other hardening measures to strengthen cluster security beyond default configurations.

Mitigation:

- Use Kyverno or any admission controller to validate and block these configurations.
- Audit against CIS Benchmarks to find gaps.

K02: Overly Permissive Authorization Configurations

Common RBAC Mis-configuration:

- Using cluster role
- Using in-build Kubernetes clusterrole
- LIST & WATCH rights to secrets
- Using ESCALATE, BIND and IMPERSONATE permissions
- Cluster Aggregation
- Third part RBAC
- Patch on namespace

Mitigation

- Use least priveledge access
- Just in time token (JWT) and short lived OIDC token

Automation tool to help configure JWT appproch... Kyverno can scan cluster and show what are the

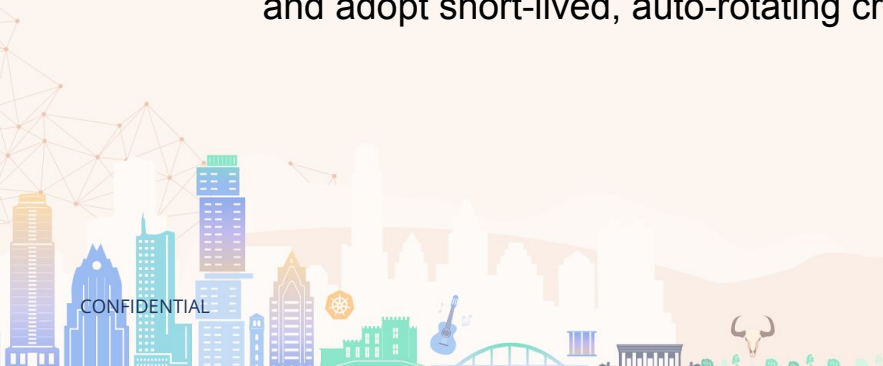
K03: Secrets Management Failures

Description

- Improper handling of secrets in images, ConfigMaps, environment variables, logs, or overly permissive RBAC can lead to credential exposure and unauthorized access.

Mitigation:

- Use Kubernetes Secrets or external secret managers, enforce strict RBAC, enable encryption, and adopt short-lived, auto-rotating credentials with workload identity.



CONFIDENTIAL



K04: Lack Of Cluster Level Policy Enforcement

Description

- Lack of centralized policy enforcement can lead to insecure configurations, inconsistent security posture, unrestricted lateral movement, and compliance gaps across Kubernetes clusters.

Mitigation:

- Enforce cluster-wide security controls using Pod Security Admission (PSA), Validating Admission Policies, and policy engines like OPA/Gatekeeper, Kyverno, or Kubewarden.

K05: Missing Network Segmentation Controls

Description

- Kubernetes allows unrestricted pod-to-pod communication by default, increasing the risk of lateral movement, control plane access, and data exfiltration after a workload compromise.

Mitigation:

- Implement namespace-level NetworkPolicies with a default-deny approach using tools like Calico, Cilium, or Kyverno, and minimize the use of host networking.

K06: Overly Exposed Kubernetes Components

Description

- Publicly exposed Kubernetes APIs, Kubelets, or etcd instances increase the risk of unauthorized access, information leakage, and full cluster compromise.

Mitigation:

- Restrict API and Kubelet access to trusted networks using private endpoints, firewall rules, and zero-trust access controls with tools like Teleport or Tailscale.
- Continuously detect exposed or misconfigured components using CNCF and OSS security tools like Kubescape, kube-bench, and Falco.

K07: Misconfigured And Vulnerable Cluster Components

Description

- Enforce least-privilege cloud access using workload identities such as IRSA (EKS), Workload Identity (GKE/AKS), and Misconfigured or unpatched Kubernetes components, container runtimes, and cluster nodes can expose clusters to privilege escalation, unauthorized access, and known vulnerabilities.

Mitigation:

- Harden Kubernetes components and worker nodes using CIS Benchmarks and continuously validate configurations with tools like Kubescape and kube-bench.
- Regularly patch Kubernetes and underlying node operating systems, and use hardened container-focused OS distributions such as Talos Linux, Flatcar Container Linux, or Bottlerocket.

K08: Cluster To Cloud Lateral Movement

Description

- Kubernetes workloads can be abused to move laterally into cloud infrastructure through overly permissive node IAM roles, exposed instance metadata services, or leaked static credentials, potentially leading to full cloud account compromise.

Mitigation:

- Enforce workload-specific cloud identities using IRSA (EKS), Workload Identity (GKE/AKS), and eliminate static credentials in favor of short-lived, scoped access via ServiceAccounts.
- Block access to instance metadata services using NetworkPolicies, harden IMDS (e.g., IMDSv2), and continuously detect suspicious behavior with runtime security tools like Falco and Tetragon.

K09: Broken Authentication Mechanisms

Description

- Weak authentication mechanisms in Kubernetes (e.g., long-lived certificates, anonymous access, shared or default service account tokens) can allow attackers to impersonate legitimate users or workloads and gain unauthorized entry to the API server.

Mitigation:

- Strengthen identity verification by enforcing OIDC/JWT-based authentication with short-lived, externally managed credentials and MFA, while disabling anonymous access and default service account token usage.

K10: Inadequate Logging And Monitoring

Description

- Inadequate logging and monitoring in Kubernetes can result in missed security events, making it difficult to detect attacks, investigate incidents, or understand the full impact of a breach.

Mitigation:

- Enable and retain Kubernetes audit logs, and centralize cluster, node, and workload logs into a dedicated logging pipeline or SIEM with reliable buffering and retention for forensic visibility and alerting.

Thankyou!

Connect with me here:

